

신경망을 활용한 데이터 적합

6장에서 다루는 내용

- 선형 모델과 비교하여 비선형 활성화 함수의 차이점
- 파이토치의 nn 모듈 다루기
- 신경망을 활용한 선형 적합 문제 풀기

지금까지 선형 모델이 어떻게 학습하며 파이토치로 구현했을 때 내부에서 어떤 일이 일어나는지를 살펴보았다. 앞서 다룬 예제는 하나의 입력과 하나의 출력을 가지는 선형 모델을 사용한 매우 단순한 회귀 문제였다. 단순한 예제 덕분에 모델 자체의 구현보다 모델이 어떻게 학습하는지에 집중할 수 있었다. 전체 과정을 보여주는 그림 5.2에서 살펴보았듯이(그림 6.1에서 다시 보여줄 것이다) 모델을 훈련시키는 상위 개념을 이해하기 위해 모델의 세부 내용은 필요하지 않았다. 파라미터에 오차를 역전파하고 손실값에 근거한 기울기로 파라미터를 조정하는 과정은 어떤 모델을 사용하더라도 동일했다.

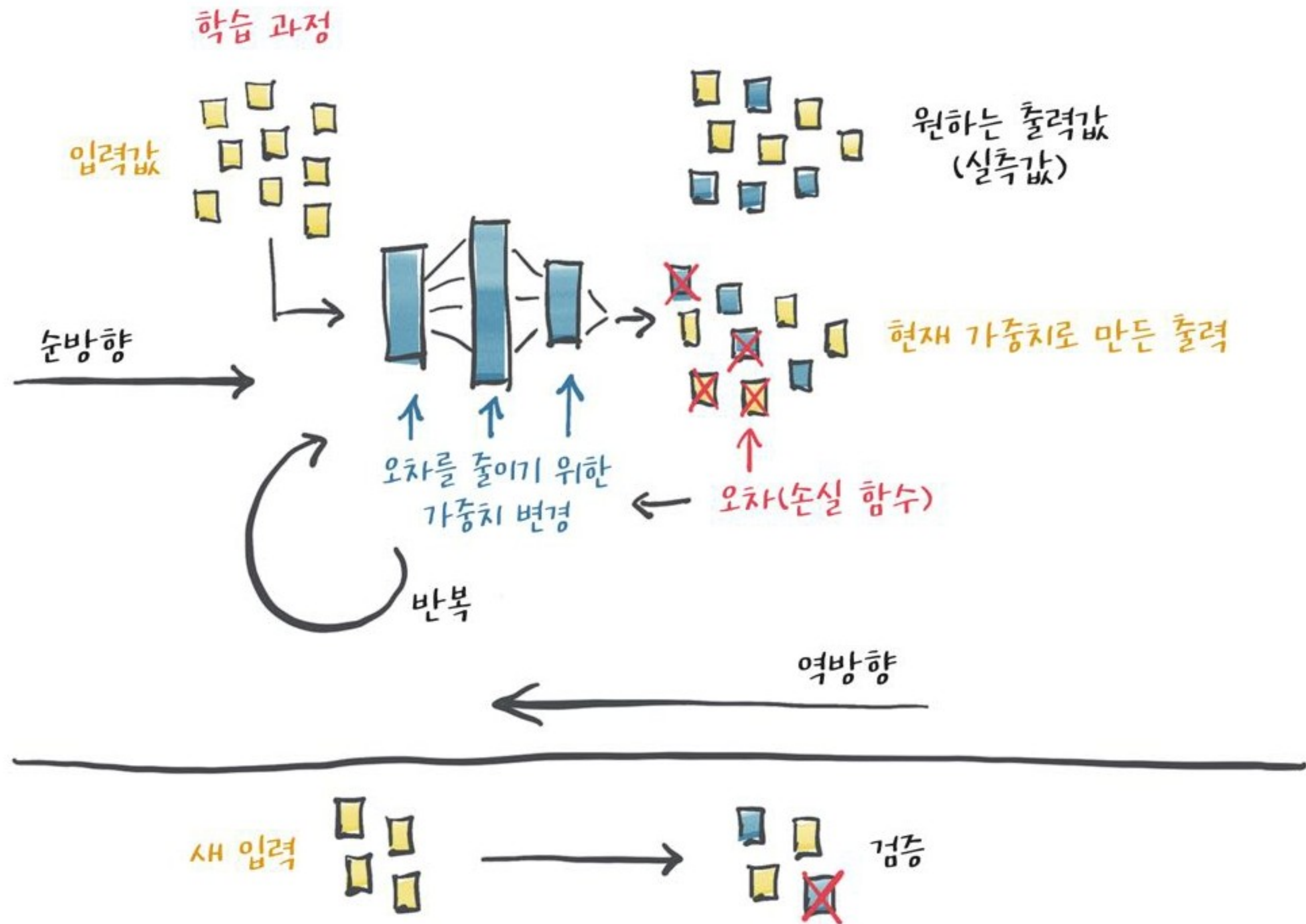


그림 6.1 5장에서 구현한 학습 과정의 멘탈 모델

이번 6장에서는 모델 아키텍처를 살짝 바꿔본다. 온도 변환 문제를 풀기 위한 완전한 인공 신경망을 구현할 예정이다. 앞서 5장에서 다룬 훈련 루프나, 훈련셋과 검증셋으로 나눈 화씨 섭씨 샘플은 그대로 사용한다. 2차 방정식 모델로 시작해보자. `model`을 입력에 대한 2차 함수로 다시 작성하자(예를 들면 $y = a * x^2 + b * x + c$). 이런 모델은 미분 가능하므로 파이토치가 기울기 계산을 처리해주며 훈련 루프는 이전처럼 동작할 것이다. 우리는 함수 모양을 계속 고칠 예정이므로 모델을 위한 수식은 그다지 흥미롭지 않을 것이다.

이번 장에서는 우리가 지금까지 다룬 기본 작업들을, 앞으로 여러분이 프로젝트를 수행할 때마다 매일같이 사용할 파이토치의 기능으로 구현해볼 것이다. 이제 흑마법처럼 신비해 보이던 파이토치 API의 내부에서 어떤 일이 일어나는지 이해할 수 있을 것이다. 새 모델을 구현하기 전에 먼저 **인공 신경망** `artificial neural network`이라는 용어부터 짚고 넘어가자.

6.1 인공 뉴런

딥러닝의 핵심인 신경망은 복잡한 함수를 단순한 함수들의 합성으로 표현할 수 있는 수학적 엔티티다. 신경망^{neural network}이라는 용어는 인간의 뇌가 동작하는 방식을 연상시킨다. 사실을 말하자면, 초기 모델들은 신경과학^{neuroscience} 분야의 영향을 받긴 했지만 오늘날의 인공 신경망은 실제 뇌에 있는 뉴런의 동작과는 조금만 비슷하다.¹ 인공 신경망과 사람의 신경망은 복잡한 함수에 근사하기 위해 어느 정도 유사한 수학적 접근을 보이며 이 전략은 매우 효과적으로 동작한다.

노트

이제부터는 ‘인공’이라는 말을 떼어놓고 그냥 ‘신경망’이라고 얘기해 보겠다.

이 복잡한 함수의 기본 빌딩 블록은 그림 6.2에서 볼 수 있는 뉴런^{neuron}이다. 본질적으로 뉴런은 단순히 입력에 대한 선형 변환과(예를 들어 입력에 특정 값(가중치)을 곱하고 상수(편향값)를 더한다) 활성화 함수^{activation function}라 부르는 고정된 비선형 함수를 적용하는 역할을 한다.

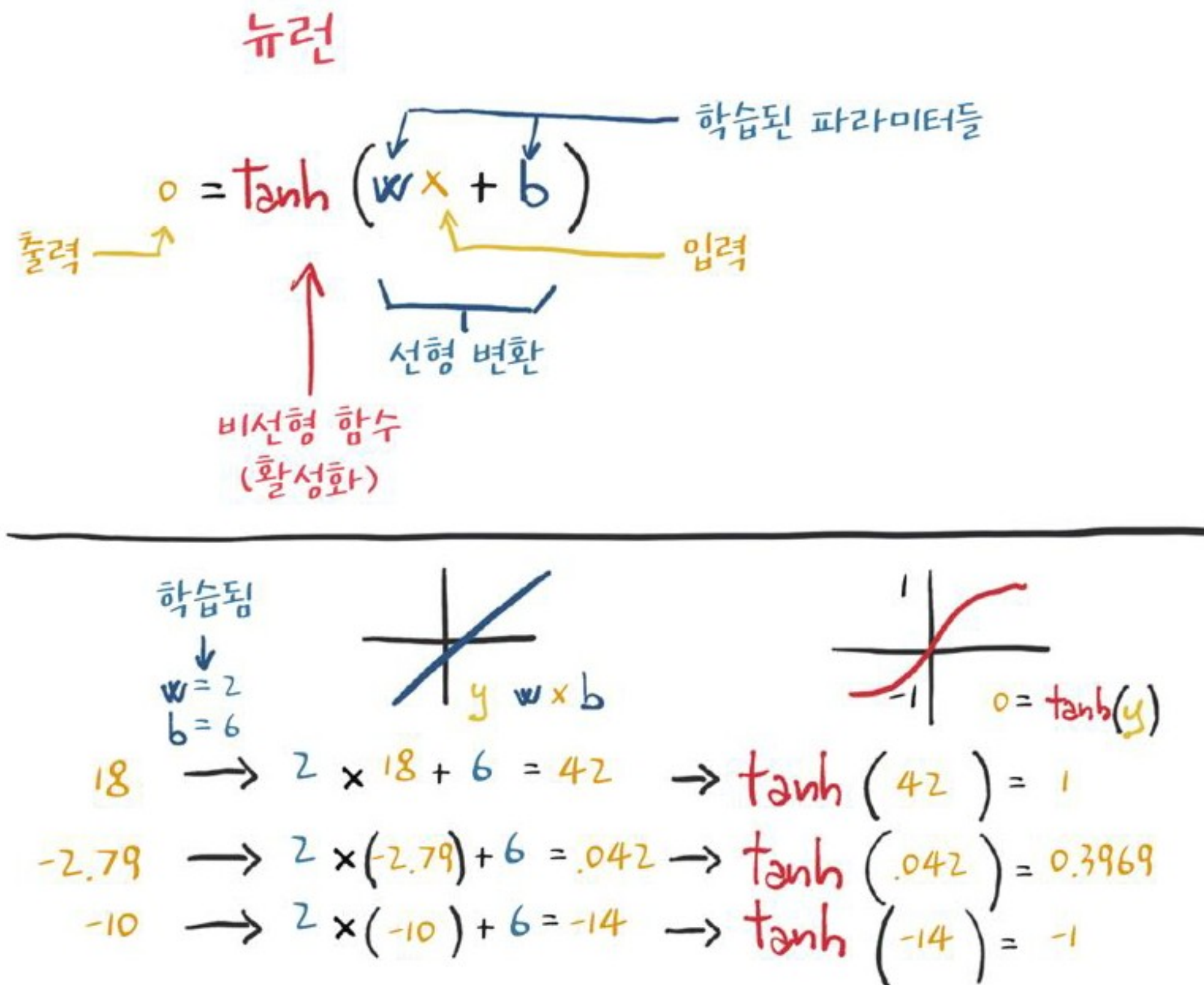


그림 6.2 인공 뉴런: 비선형 함수로 둘러싸인 선형 변환

수학적으로는 $o = f(w * x + b)$ 로 적을 수 있다. x 는 입력, w 는 가중치 혹은 비율 인자, 그리고 b 는 편향값 혹은 오프셋이라 부른다. f 는 활성화 함수인데 쌍곡탄젠트(hyperbolic tangent)로 설정했고, 그림에서는 $\tanh$ 함수에 해당한다. 일반적으로 x 나 o 는 단순 스칼라값 혹은 (여러 스칼라 값을 가진) 벡터이고, 비슷하게 w 는 단일 스칼라 혹은 행렬이며, b 는 스칼라나 벡터일 수 있는데 벡터인 경우에는 입력 및 가중치와 차원이 일치해야 한다. 벡터의 경우에는 여러 차원으로 가중치와 편향 값을 가진 여러 개의 뉴런을 나타내므로, 이런 표현식은 뉴런 계층(layer)이라 한다.

6.1.1 다중 계층 신경망 합성하기

그림 6.3의 다중 계층 신경망은 지금까지 다룬 함수들의 합성으로 만들어졌다.

$$\begin{aligned}x_1 &= f(w_0 * x + b_0) \\x_2 &= f(w_1 * x_1 + b_1) \\&\dots \\y &= f(w_n * x_n + b_n)\end{aligned}$$

여러 뉴런으로 구성된 계층의 출력을 이어지는 다음 계층의 입력으로 사용한다. 여기에서 w_0 는 행렬이고 x 는 벡터임을 주의하자! 벡터를 사용하기 때문에 w_0 가 하나의 가중치 값이 아닌 계층에 있는 뉴런 전체에 대한 가중치를 담고 있다.

6.1.2 오차 함수

이전의 선형 모델과 앞으로 사용할 딥러닝과의 주요한 차이점 중 하나는 오차 함수의 모양이다. 선형 모델과 오차를 제공한 손실 함수는 볼록 함수 형태의 오차 곡선을 가지고 있으므로, 최솟값이 분명하게 하나로 정의된다. 다른 메소드를 사용한다고 해도 오차 함수를 최소화시키는 파라미터를 자동으로 확실하게 찾을 수 있다. 바꿔 말하면 파라미터 조정이 하나의 정답에 가까워지도록 값을 추정한다는 의미다.

반면 신경망은 동일한 오차 제곱 손실 함수를 사용한다 하더라도 볼록 함수의 오차 곡선의 모양이 이와 다르다! 우리가 근사하려는 각 파라미터는 정해진 답이 없다. 대신 모든 뉴런이 협력하여 유용한 출력을 만들기 위한 파라미터 획득을 목표로 할 뿐이다. 유용한 출력이라는 표현은 다른 말로 하면 참값에 근사한다는 의미로 어느 정도는 불완전함을 뜻한다. 어디에서 어떻게 불완전성이 나타나는지는 임의적이라서, 출력을 제어하는 파라미터 역시 임의적인 측면이 있다. 그래서 한편으로는 신경망 훈련이 기계적인 파라미터 추정과 유사해 보이지만 이론적인 기반은 전혀 다르다는 사실을 알아두자.

신경망의 오차 함수가 볼록 형태가 아닌 것은 활성화 함수 때문이다. 매우 다양한 함수에 근사할 수 있는 뉴런의 앙상블 능력은 각 뉴런이 내포하는 이러한 선형 그리고 비선형적 조합 덕분이다.

6.1.3 우리에게 필요한 건 활성화 함수

앞에서 본 것처럼 (심층) 신경망의 가장 단순한 형태는 (스케일링과 오프셋이 더해진) 선형 연산에 활성화 함수가 이어진 형태다. 앞에서 모델에 선형 연산이 있었는데 이게 모델의 전부였다. 활성화 함수는 여기에서 두 가지 중요한 역할을 한다.

- 활성화 함수는 모델 내부에서 출력 함수가 값마다 다른 기울기를 가지도록 만들어 주는데, 이는 선형 함수 자체로는 불가능하다. 신경망은 여러 출력에 대해 각기 기울기가 다른 특성을 절묘하게 합성하여, 6.1.6절에서 다룬 여러 임의의 함수에 근사할 수 있게 된다.²
- 신경망의 마지막 계층은 이전의 선형 연산 출력을 주어진 범위로 모으는 역할을 한다.

두 번째 역할에 대해 살펴보자. 각 그림에 ‘똥똥한 땀땀이’ 점수를 매긴다고 가정하자. 리트리버와 스페니얼은 높은 점수를 얻지만 비행기나 쓰레기차는 낮은 점수를 얻는다. 곰 사진 역시 쓰레기차보단 높지만 낮은 점수를 얻어야 한다.

문제는 ‘높은 점수’가 어느 정도인지 정의해야 한다는 점이다. float32의 모든 범위를 사용할 수도 있지만 이러면 점수가 너무 높아진다. 간단하게 “10점 만점”으로 정의해도 모델은 10점을 넘어 11점을 만들 수도 있다. 내부에 $(w * x + b)$ 행렬곱과 행렬합이 존재하고 이런 연산의 출력값이 알아서 특정 범위로 제한되지는 않는다는 점을 기억하자.

■ 출력 범위 제한

우리는 선형 연산의 출력을 특정 범위로 제한해서 이 출력을 사용하는 곳에서 강아지가 12/10점, 곰이 -10점, 쓰레기차가 -1,000점이 나와서 별도 처리하는 경우가 없게 해야 한다.

가능한 방법 중 하나는 출력값을 제한하는 것이다. 0 이하는 무조건 0으로 하고 10 이상은 무조건 10으로 두면 된다. 이는 `torch.nn.Hardtanh`라는 단순한 활성화 함수다(<https://pytorch.org/docs/stable/nn.html#hardtanh>, 기본값이 -1에서 +1이다).

■ 출력 범위 줄이기

이와 유사하며 잘 동작하는 함수 중 하나로 `torch.nn.Sigmoid`가 있다. 여기에는 $1 / (1 + e^{-x})$ 와 `torch.tanh`, 그리고 조금후에 살펴볼 여러 함수들이 포함된다. 이들 함수는 x 가 음의 무한대로 가면 0이나 -1에 점차 가까워지고 양의 무한대로 갈수록 1에 가까워지는 곡선을 가지며 $x = 0$ 일 때에는 상수 기울기를 가진다. 개념적으로 이런 모양의 함수는 매우 잘 동작한다. 뉴런이 민감하게 반응하는 함수의 중앙 영역에 선형 함수 형태가 존재하고 다른 영역은 바로 경계값에 가까워지기 때문이다. 그림 6.4에서 보다시피 쓰레기차의 값은 -0.97이며, 곰과 여우, 늑대는 -0.3에서 0.3 사이 구간의 값을 가진다.

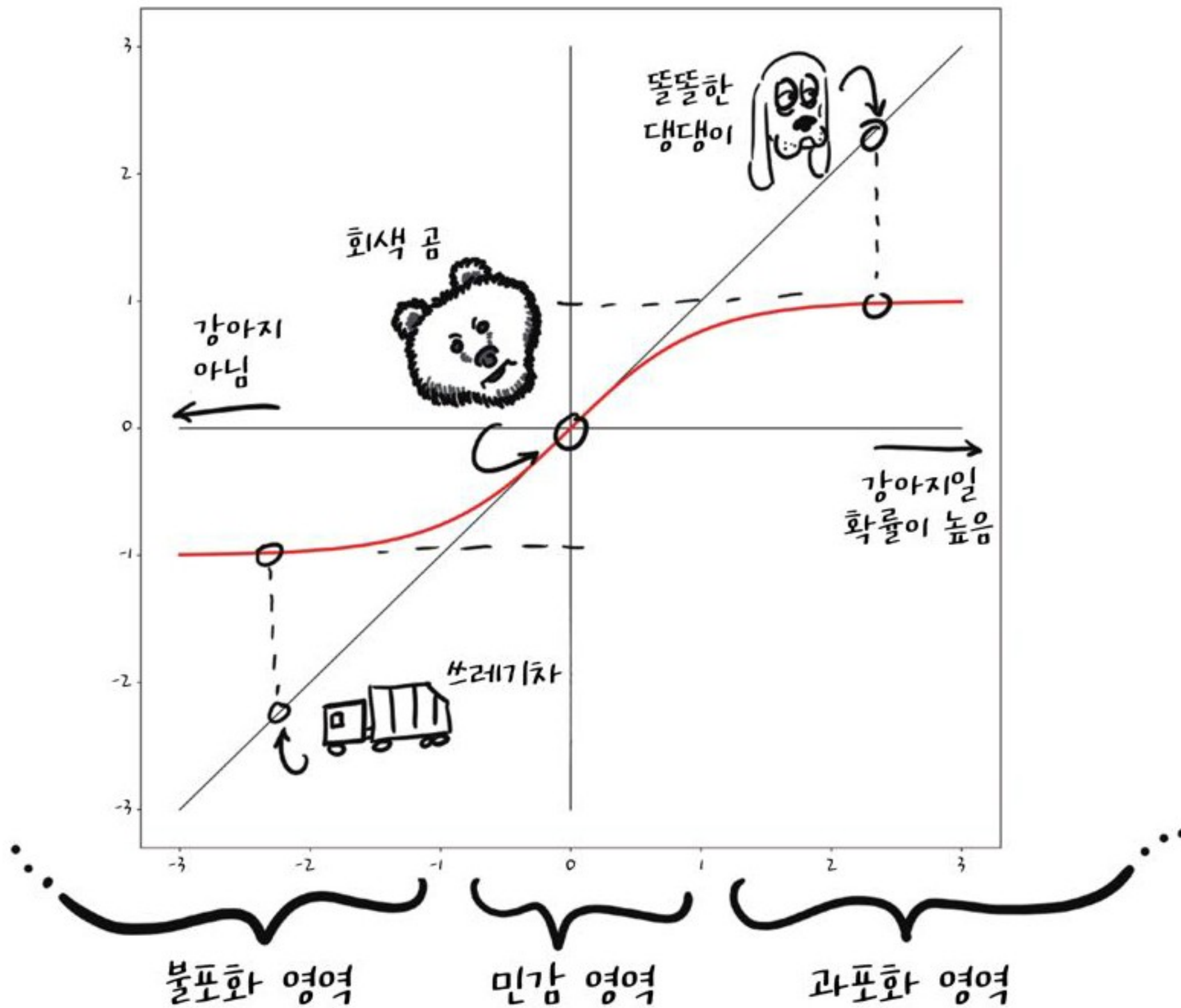


그림 6.4 tanh 활성 함수를 사용하여 각기 강아지, 곰, 쓰레기차가 얼마나 강아지와 비슷한지를 매핑한 그림

이로 인해 쓰레기차는 ‘강아지 아님’이 되고 푹푹한 땡땡이는 ‘강아지가 확실함’이 되는 동시에
곰은 이 둘 가운데쯤에 놓인다. 코드를 통해 정확한 값을 볼 수 있다.

```
>>> import math
>>> math.tanh(-2.2)  ← 쓰레기차
-0.9757431300314515
>>> math.tanh(0.1)  ← 곰
0.09966799462495582
>>> math.tanh(2.5)  ← 푹푹한 땡땡이
0.9866142981514303
```

민감 영역에서 곰은 작은 변화만으로도 결과에 큰 영향을 준다. 예를 들어 회색 곰을 개의 얼굴
과 좀 더 비슷한 북극곰으로 바꾸면, 그래프상에서 ‘강아지일 확률이 높음’ 영역에 해당하는 곳
으로 Y축을 따라 미끄러지며 올라간다. 반대로 코알라로 바꾼다면, 활성화된 출력에서 강아지
일 확률이 낮은 쪽으로 떨어지는 것을 확인할 수 있다. 한편 쓰레기차를 강아지로 구별되게 하
기 위해 우리가 할 수 있는 일은 거의 없다. 큰 변화를 준다고 해봐야 -0.97에서 -0.8 정도로만
이동할 수 있을 뿐이다.

6.1.4 그 밖의 활성화 함수

지금까지 다룬 활성화 함수 외에도 상당수가 있는데 이 중 일부를 그림 6.5에서 볼 수 있다.

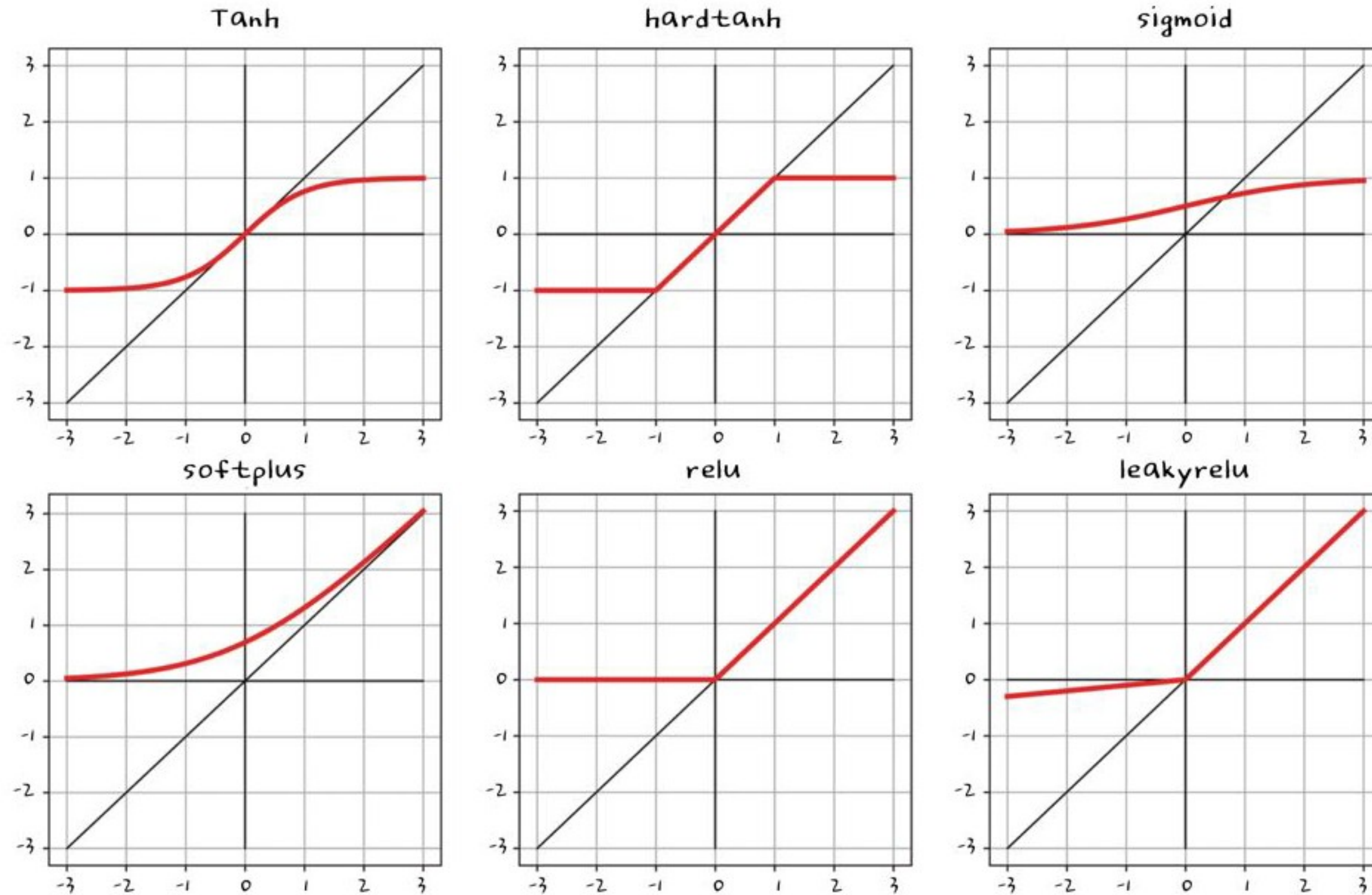


그림 6.5 일반적인 활성화 함수와 일반적이지 않은 활성화 함수 모음

첫째 열에는 매끄러운 함수^{smooth function}인 Tanh와 Softplus가 있고 두 번째 열에는 이보다 각진^{hard} 버전의 활성화 함수에 해당하는 Hardtanh와 ReLU가 있다. ReLU^{rectified linear unit}는 통상 활성화 함수들 중에서 가장 성능이 좋기 때문에 기억해둘 만하며, 최신 연구 결과에 많이 사용된다. 시그모이드^{Sigmoid} 활성화 함수는 로지스틱^{logistic} 함수라고도 하는데 초기 딥러닝에서 많이 사용했으나 이제는 출력값이 확률적일 때 값을 0에서 1 사이로 옮기려는 경우 외에는 많이 사용하지 않는다. 마지막으로 LeakyReLU는 표준 ReLU를 변형하여 입력이 음수일 때 출력을 0 대신 약간의 양의 기울기를 가지게 만든 것이다(통상 기울기가 0.01이지만 그림에서는 잘 보이도록 0.1로 했다).

6.1.5 최적의 활성화 함수 고르기

실제에서 효과가 증명된 다양한 함수가 그림 6.5보다 더 많기 때문에 활성화 함수는 흥미로운 영역이고 제한도 별로 없다. 어떤 경우에는 명백히 틀렸다고 입증될 수도 있지만 활성화 함수의 일반적인 특성은 다음과 같다. 정의³에 따르면 활성화 함수는 다음과 같은 특성을 지닌다.

- 비선형이다. 활성화 함수 없이 ($w * x + b$)를 반복해서 적용하면 동일한 (아핀 선형) 형태의 함수가 된다. 활성화 함수의 비선형성이 전체 신경망이 더 복잡한 함수로 근사하도록 해주는 역할을 한다.
- 미분 가능하다. 덕분에 기울기 계산이 가능해진다. Hardtanh나 ReLU에서 볼 수 있는 불연속점은 큰 문제가 되지 않는다.

이런 특성이 없다면 신경망은 단순한 선형 모델이 되거나 훈련시키기 어려워진다.

활성 함수에 대해 다음은 맞는 말이다.

- 최소한 하나의 민감한 구간을 가지며 입력에서 중요 범위가 변경되면 일치하는 출력 영역에서도 중요한 변화가 발생한다. 이 부분이 훈련에서 필요한 구간이다.
- 대부분이 둔감한(포화된) 구간을 가지며 이 구간에서는 입력의 변화가 출력에 거의 영향을 주지 않거나 아예 변화가 없다.

예를 들면 Hardtanh 함수는 일부 영역에 대해서 입력에 여러 가중치와 편향값을 조합하여 구간적인 선형 근사를 만드는 데 사용할 수 있다.

항상은 아니지만 활성 함수는 다음과 같은 특징 중 하나 이상을 가진다.

- 입력이 음의 무한대로 갈 때 만나거나 가까워지는 저점이 있다.
- 비슷하게 양의 무한대로 갈 때에도 고점이 있다.

역전파가 어떻게 일어나는지를 생각해보면 입력이 응답 범위에 있을 때 오차가 활성화 단계를 통해 더 효과적으로 역방향으로 전파되어야 함을 이해할 수 있다. 반대로 입력이 포화된 경우에는 오차가 뉴런에 큰 영향을 미치지 않아야 한다(출력값의 평평한 부분으로 인해 기울기가 0에 가까워지는 상황).

이런 특성들을 모두 합치면 매우 강력한 메커니즘이 만들어진다. 선형 그리고 활성화 유닛으로 만들어진 신경망에 다른 입력이 나타나면 (a) 같은 입력이라도 반응하는 유닛이 다르고 각 유닛 별로 다른 범위로 반응하며 (b) 학습 과정에서도 다른 유닛은 영향받지 않고 입력에 대해 민감 범위를 가지고 있는 관련된 유닛만 오차값에 의해 영향을 받는다. 부가적으로 입력에 대해 활성화된 곳에서의 미분은 1에 가까운 값을 가지는 경우가 빈번한데 앞에서 본 선형식에 맞춰가는 모습처럼 이런 성질을 통해 반응하는 유닛의 범위로부터 경사 하강으로 선형 변환식의 파라미터를 추정할 수 있다.

이제 우리는 어떻게 해서 많은 선형 + 활성화 유닛 조합이 병렬로 연결된 후, 스택처럼 하나씩 쌓여서 복잡하고 정교한 함수에 근사할 수 있는 수학적 객체가 되는지를 좀 더 깊게 이해해보려 한다. 유닛을 어떻게 조합하느냐에 따라 입력에 대해 다른 범위로 응답하며 유닛들이 가지는 파라미터의 최적화는 출력값이 포화될 때까지 선형 함수의 학습처럼 동작하기 때문에 생각보다 쉽게 경사 하강으로 최적화된다.

6.1.6 신경망에서 학습의 의미

선형 변환을 스택처럼 쌓아 만든 빌딩 블록에 미분 가능한 활성화 함수를 연결한 모델은 고차원의 비선형 작업에도 근사하며 파라미터 역시 경사 하강을 통해 꽤 잘 추정할 수 있다. 이와 같은 특징은 모델이 수백만 개의 파라미터를 다루는 경우에도 마찬가지다. 심층 신경망이 매력적인 이유 중 하나는 데이터를 표현하는 정확한 함수에 대해 크게 고민할 필요가 없다는 점이다. 이차 함수일지 혹은 구간적 다항식 아니면 다른 함수일지는 생각하지 않아도 된다. 심층 신경망 모델로 인해 우리는 범용의 근사 함수와 파라미터 추정 수단을 가지는 셈이다. 근사 함수는 필요에 따라 단순 빌딩 블록을 구성하는 것만으로 모델의 용량이나 얼마나 복잡한 입출력 관계를 모델링할지 커스터마이징할 수 있다. 그림 6.6에서 이와 관련된 다양한 예제를 볼 수 있다.

좌상단부 네 개의 A, B, C, D 그래프는 각각 네 개의 뉴런을 나타내며 임의의 가중치와 편향값을 가진다. 각 뉴런은 -1에서 1 사이의 값을 가지는 Tanh 활성화 함수를 사용하는데 다양하게 주어진 가중치와 바이어로 인해 중심으로부터 얼마만큼 이동하는지와 최소에서 최대값 사이에 어떻게 급격하게 전환되는지를 바꾸더라도 모두가 기본적인 형태는 유지한다. 제일 오른쪽 열은

$A + B$ 나 $C + D$ 처럼 두 개의 뉴런을 합쳤을 때를 보여준다. 이 부분에서 단일 뉴런 계층을 흉내내는 흥미로운 특성이 보이기 시작한다. $A + B$ 는 약한 S 커브를 보여주고 양 극단에 대해 0으로 수렴하며 중앙에는 위아래로 볼록한 지점이 있다. 이와 대조적으로 여섯 번째의 $C + D$ 그래프는 위로 볼록한 지점이 하나 있는데, 보통 한 개의 뉴런이 최댓값 1을 가지는 데 반해 그 보다 더 높이 솟아 있다.

마지막 줄에는 두 개의 계층을 가지도록 뉴런을 합성했다. $C(A + B)$ 와 $D(A + B)$ 모두 $A + B$ 같은 위아래 방향의 볼록 지점을 가지지만 위쪽으로 볼록한 정도가 약하다. $C(A + B) + D(A + B)$ 합성은 조금 새로운데, 아래로 볼록한 지점은 둘, 위로 살짝 볼록한 지점은 하나가 있으며, 관심 영역은 중앙에서 약간 왼쪽으로 치우쳐 있다. 이 모든 그래프는 기껏해야 두 개의 계층에서 많아야 네 개의 뉴런을 사용했을 뿐이다!

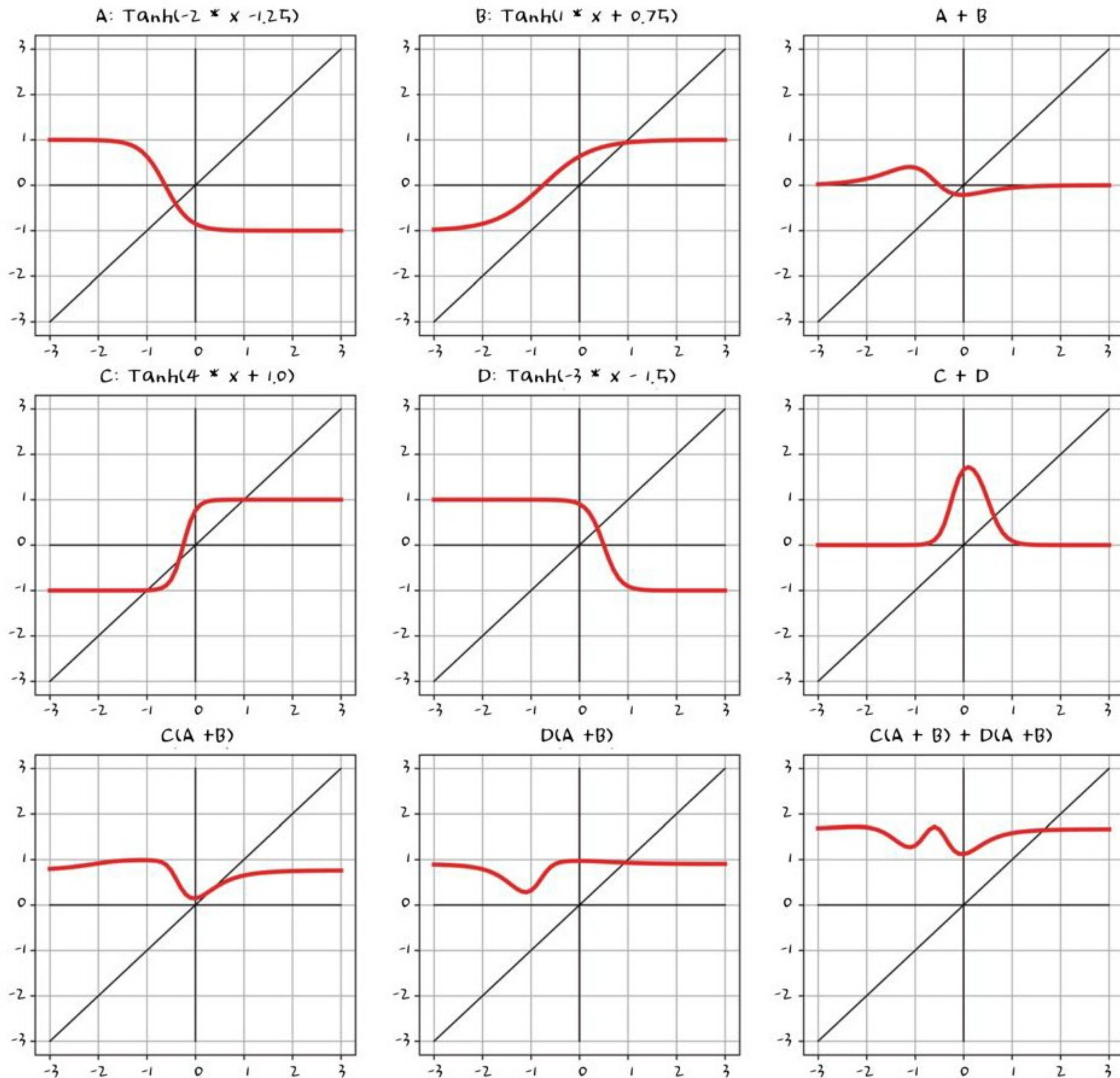


그림 6.6 비선형 출력을 위해 선형 유닛과 tanh 활성화 함수를 합성한 예

다시 말하지만 그림에서의 뉴런 파라미터 값은 결과를 재미있게 하기 위해 임의로 정한 것이다. 실제 훈련은 지리 좌표와 연도를 주면 기온을 예측하는 신경망이 작업을 정확하게 수행하기 위해 필요한 가중치와 편향값을 찾는 과정으로 구성된다. **작업을 성공적으로 수행할 수 있다는 것은** 훈련 데이터로 출력을 만드는 과정과 동일하게 기존에 입력되지 않았던 데이터를 넣었을 때 정확한 출력값을 얻는다는 의미다. 잘 훈련된 신경망은 데이터에 내재된 의미를 숫자의 형태로 가중치와 편향값에 넣어 이전에는 보지 못했던 데이터라도 정확하게 동작한다.

이제 학습 메커니즘에 대해 한걸음 더 심도 있게 이해해보자. 심층 신경망은 고수준의 비선형적인 현상에 대해 명시적인 모델 없이 근사할 수 있는 능력을 우리에게 부여한다. 명시적인 모델 대신 훈련되지 않은 일반 모델에서 출발하며 우리는 여러 입출력쌍 예제와 역전파할 손실 함수를 제공하여 일반 모델을 특정 작업에 최적화한다. 예제를 통해 일반 모델을 최적화하는 과정을 **학습**learning이라 부른다. 모델 자체는 각각의 작업을 염두에 두고 만들어진 것이 아니다. 일반 모델에는 특정 작업을 수행하는 방법을 기술하는 규칙이 없다.

온도계 예제에서 우리는 두 온도계가 온도를 선형적으로 측정한다고 가정했다. 그리고 암묵적으로 이 가정을 따르는 규칙을 인코딩했으며, 입출력 함수의 형태를 하드코딩했다. 이 때문에 선 근처에 나열되지 않는 데이터 포인트는 근사할 수 없었다. 문제의 차원이 증가하고(입출력이 많으며) 입출력 관계가 복잡해질수록 이런 식의 입출력 함수 가정은 잘 동작하지 않을 것이다. 물리학자나 응용수학자는 기본 원칙에서 어떤 현상을 설명하는 함수를 먼저 만들고 측정을 통해 그 함수의 파라미터 값을 추정해서 세계에 대한 정확한 모델을 만들어내는 일이 빈번하다. 반대로 심층 신경망은 우리에게 현상을 설명할 만한 모델을 만들어달라고 요구하는 대신, 충분히 다양한 입출력 관계에 대해 근사할 수 있는 다양한 함수군으로 구성된다. 덕분에 현상을 설명하는 모델을 고민하는 일은 내려놓고, 대신 얼마나 더 복잡 미묘한 문제들을 다룰 수 있을지를 생각한다. 명시적인 모델을 우리가 직접 만들기에는 능력이나 정보 혹은 계산하는 데 한계가 있으므로 이렇게 데이터에 기반한 방법 외에는 풀 수 있는 방법이 없다.

6.2 파이토치 nn 모듈

신경망에 대해 파고들수록 파이토치를 사용해서 바닥부터 만들어보고 싶은 생각이 간절해질 것이다. 이를 위한 첫 단계는 우리의 선형 모델을 신경망 유닛으로 바꾸는 것이다. 문제 자체가 선형 함수로 풀리는 것을 알고 있기 때문에 정확도는 다소 떨어질 텐데 이렇게 바꿀 필요가 있겠냐고 반문할지 모르지만 배워가는 차원에서 단순하게 시작해서 나중에 규모를 키워가는 것이 유익하다.

파이토치에는 `torch.nn`이라는 신경망 전용 서브모듈이 있다. 이 모듈에는 모든 신경망 아키텍처를 만들 수 있는 빌딩 블록이 들어 있다. 파이토치에서는 이런 빌딩 블록을 **모듈** `module`이라 부른다(다른 프레임워크에서는 계층 `layer`이라고 일컫는다). 파이토치 모듈은 `nn.Module` 베이스 클래스에서 파생된 파이썬 클래스다. 모듈은 하나 이상의 `Parameter` 객체를 인자로 받는데, 이는 텐서 타입이며 훈련 과정을 통해 값이 최적화된다(선형 모델에서의 `w`와 `b`로 보면 된다). 모듈은 다시 하나 이상의 (`nn.Module`의 서브클래스인) 서브 모듈을 속성으로 가지며 파라미터 추적이 가능하다.

노트

서브 모듈은 list나 dict 객체에 들어간 형태가 아닌 최상위 레벨 **속성**이어야 한다! 그렇지 않으면 옵티마이저가 서브 모듈(파라미터)을 찾지 못한다. 모델이 리스트나 딕셔너리 형태의 서브 모듈을 요구하는 경우에 대응할 목적으로 파이토치는 `nn.ModuleList`와 `nn.ModuleDict`를 제공한다.

당연하게도 `nn.Module`에는 `nn.Linear`라는 서브클래스가 있다. 이 클래스는 우리가 온도계 실험에서 구현한 것과 같이 (파라미터 속성인 `weight`와 `bias`를 통해서) 입력에 대한 아핀^{Affine} 변환을 적용해준다. 앞서 만든 코드를 `nn`을 사용하는 형태로 변환해보자.

6.2.1 forward 대신 `__call__` 사용하기

파이토치가 제공하는 `nn.Module`의 모든 서브클래스에는 `__call__` 메소드가 정의되어 있다. 이를 통해 `nn.Linear`를 인스턴스화하여 마치 함수인 것처럼 실행할 수 있다([code/p1ch6/1_neural_networks.ipynb](#)).

```
# In[5]:
import torch.nn as nn

linear_model = nn.Linear(1, 1)  ← 생성자 인자는 잠시 후에 살펴본다.
linear_model(t_un_val)

# Out[5]:
tensor([[0.6018],
        [0.2877]], grad_fn=<AddmmBackward>)
```

여기에서 인자를 가지고 `nn.Module` 인스턴스를 호출하면 동일한 인자로 `forward`를 호출한다. `forward` 메소드는 순방향 연산을 수행하는 반면 `__call__`은 `forward`를 호출하기 전후에 몇 가지 중요한 작업을 수행한다. 그래서 `forward`를 직접 호출할 수 있고 `__call__`과 같은 결과를 얻겠지만, 유저 코드에서는 사용하지 않는 편이 좋다.

```
y = model(x)  ← 맞음!
y = model.forward(x)  ← 티도 안 나는 오류. 이렇게 하지 말자!
```

Module.__call__의 구현은 다음과 같다(이해를 돕기 위해 JIT 관련 부분은 생략하고 단순화했다.
torch/nn/modules/module.py 483행의 Module 클래스).

```
def __call__(self, *input, **kwargs):
    for hook in self._forward_pre_hooks.values():
        hook(self, input)

    result = self.forward(*input, **kwargs)

    for hook in self._forward_hooks.values():
        hook_result = hook(self, input, result)
        # ...

    for hook in self._backward_hooks.values():
        # ...

    return result
```

코드를 보면 알겠지만 직접 .forward(...)를 호출하면 제대로 수행되지 않을 혹^{hook}들이 엄청 많다.

6.2.2 다시 선형 모델로

다시 선형 모델로 돌아가보자. `nn.Linear` 생성자는 세 개의 인자를 받는다. 입력 피처의 수와 출력 피처의 수 그리고 선형 모델이 편향값을 포함하는지 여부(기본값은 `True`다)이다.

```
# In[5]:  
import torch.nn as nn  
  
linear_model = nn.Linear(1, 1)  ← 인자는 입력 크기와 출력 크기이며 편향값은 기본인 True다.  
linear_model(t_un_val)  
  
# Out[5]:  
tensor([[0.6018],  
        [0.2877]], grad_fn=<AddmmBackward>)
```

우리의 경우 피처 수는 모듈에 대한 입출력 텐서 크기와 같아서, 값이 각각 1, 1이다. 만일 온도와 기압을 입력받는다면 입력 피처는 둘이고 출력 피처는 하나다. 나중에 중간 규모의 모듈을 가진 복잡한 모델을 다루게 되면 피처의 수는 모델의 용량과 연관이 있음을 보게 될 것이다.

이제 하나의 입력 피처와 하나의 출력 피처를 가진 `nn.Linear` 인스턴스가 있다. 이 인스턴스는 하나의 가중치와 편향값을 요구한다.

```
# In[6]:  
linear_model.weight
```

```
# Out[6]:  
Parameter containing:  
tensor([[ -0.0674]], requires_grad=True)
```

```
# In[7]:  
linear_model.bias
```

```
# Out[7]:  
Parameter containing:  
tensor([ 0.7488], requires_grad=True)
```

이제 입력값으로 모듈을 호출할 수 있다.

```
# In[8]:  
x = torch.ones(1)  
linear_model(x)  
  
# Out[8]:  
tensor([0.6814], grad_fn=<AddBackward0>)
```

파이토치가 알아서 넘어가긴 하지만 입력이 정확하지 않은 차원임을 기억하자. 우리의 모델은 하나의 입력을 받고 하나의 출력을 만든다. 하지만 파이토치의 `nn.Module`과 서브클래스들은 한 번에 여러 개의 샘플을 다루도록 설계되었다. 여러 샘플들을 수용하기 위해 모듈 입력의 0번째 차원은 **배치**^{batch}에 들어있는 샘플의 수라고 생각하자. 4장에서 실제 데이터를 텐서로 나타내는 방법을 배울 때 이 개념을 접했다.

■ 배치 입력 만들기

nn에 있는 모든 모듈은 한 번에 여러 입력을 가진 배치^{batch}에 대한 출력을 만들도록 작성되었다. 10개의 샘플에 대해 nn.Linear를 실행해야 한다고 가정해보자. B 가 배치의 크기이고 N_{in} 이 입력 피처의 크기라면 크기가 $B \times N_{in}$ 인 입력 텐서를 만들고 모델로 실행하는 예는 다음과 같다.

```
# In[9]:
x = torch.ones(10, 1)
linear_model(x)

# Out[9]:
tensor([[0.6814],
        [0.6814],
        [0.6814],
        [0.6814],
        [0.6814],
        [0.6814],
        [0.6814],
        [0.6814],
        [0.6814],
        [0.6814]], grad_fn=<AddmmBackward>)
```


무슨 일이 일어나는지 파악하려면 배치 이미지 데이터로 비슷한 상황을 나타낸 그림 6.7을 살펴 보자. 입력은 배치 크기가 3인(강아지, 새, 자동차라고 하자) $B \times C \times H \times W$ 텐서이고 채널의 차원은 3이며(빨강과 녹색 그리고 파랑) 높이와 너비는 정의되지 않았다. 그리고 출력은 $Nout$ 이 출력 피처의 수(예제에서는 4)일 때 $B \times Nout$ 차원의 텐서다.

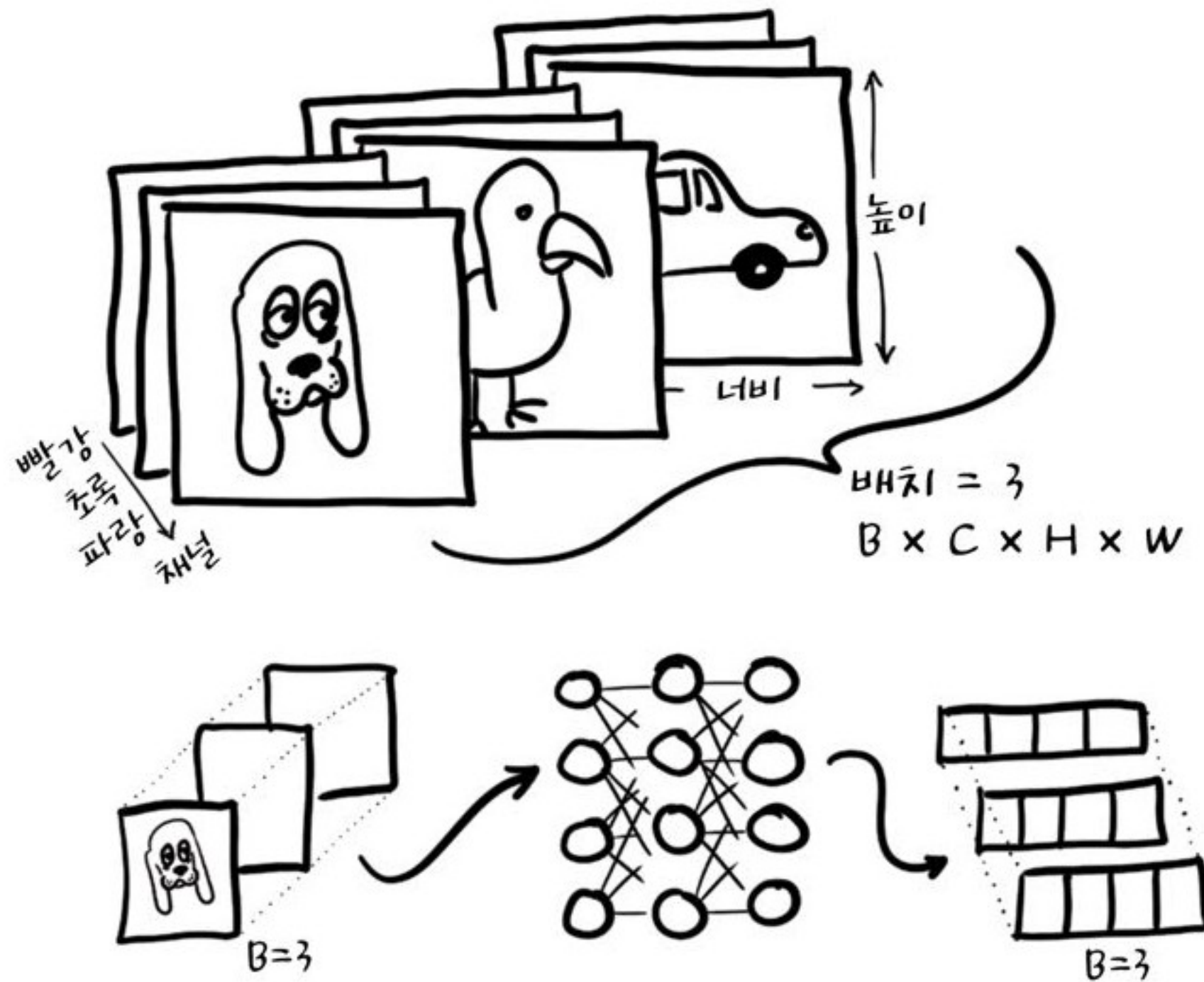


그림 6.7 배치(batch)로 묶여 신경망으로 들어가는 세개의 RGB 이미지. 출력 또한 크기가 4인 벡터 세 개가 묶인 배치다.

■ 배치 최적화

배치^{batch}를 수행하는 이유는 다양하다. 주요 이유 중 하나는 연산량을 충분히 크게 만들어 준비한 자원을 최대한 활용하기 위함이다. 병렬 연산에 최적화된 GPU에 작은 모델을 넣고 입력 하나를 수행하면, 대부분의 자원은 놀게 된다. 여러 입력을 묶어 하나를 배치로 한 번에 실행하면, 놀고 있는 다른 유닛도 계산에 사용할 수 있도록 만들고 여러 개의 입력을 처리하는 데 걸리는 시간이 하나의 입력을 처리하는 데 걸리는 시간과 동일하므로 이득이다. 추가로 어떤 고급 모델은 전체 배치에 대해 통계 정보를 사용하는데 이때 배치 사이즈가 클수록 더 나은 통계값을 얻기도 한다.

두 개의 1차원 텐서 t_u 와 t_c 로 만들어진 크기가 B 인 우리의 온도계 데이터로 돌아가보자. 브로드캐스팅 덕분에 스칼라값 w 와 b 를 그대로 사용한 $w * x + b$ 로 표현이 가능해진다. 입력 피치가 하나이기 때문에 동작한다. 만일 두 개였다면 행에는 샘플이 있고 열에는 피치가 있는 행렬로 바꾸기 위한 1차원 텐서가 추가로 필요했을 것이다.

딱 `nn.Linear`를 사용하면 되는 부분이다. B 개의 입력을 $B \times N_{in}$ 으로 리세이프하면 되며 이 경우 N_{in} 은 1이다. `unsqueeze`를 사용하면 쉽다.

```
# In[2]:
t_c = [0.5, 14.0, 15.0, 28.0, 11.0, 8.0, 3.0, -4.0, 6.0, 13.0, 21.0]
t_u = [35.7, 55.9, 58.2, 81.9, 56.3, 48.9, 33.9, 21.8, 48.4, 60.4, 68.4]
t_c = torch.tensor(t_c).unsqueeze(1)
t_u = torch.tensor(t_u).unsqueeze(1) | 1번 축에 여분의 차원을 추가한다.

t_u.shape

# Out[2]:
torch.Size([11, 1])
```

됐다. 학습 코드를 업데이트하자. 먼저 직접 작성한 모델을 `nn.Linear(1, 1)`로 바꾼 후 옵티마이저에 선형 모델 파라미터를 전달한다.

```
# In[10]:
linear_model = nn.Linear(1, 1)  ← 앞에서 나온 내용을 다시 정의한 것
optimizer = optim.SGD(
    linear_model.parameters(),  ← [params] 부분 대신 넣은 것
    lr=1e-2)
```


전에는 파라미터를 직접 만들고 `optim.SGD`의 첫 번째 인자로 전달해야 했다. 이제는 `nn.Module`이나 어떤 하위 모듈에 대해서도 `parameters` 메소드로 파라미터 리스트를 얻을 수 있다.

```
# In[11]:  
linear_model.parameters()
```

```
# Out[11]:  
<generator object Module.parameters at 0x7f94b4a8a750>
```

```
# In[12]:  
list(linear_model.parameters())
```

```
# Out[12]:  
[Parameter containing:  
tensor([[0.7398]], requires_grad=True), Parameter containing:  
tensor([0.7974], requires_grad=True)]
```

`parameters` 메소드를 호출하면, 모듈의 `init` 생성자에 정의된 서브 모듈까지 재귀적으로 호출하며 만나는 모든 파라미터 리스트를 담은 리스트를 반환한다. 이 값을 앞에서처럼 그대로 옵티마이저 생성자에 전달할 수 있다.

이쯤이면 훈련 루프에서 어떤 일이 일어나는지 알 수 있을 것이다. 옵티마이저에는 텐서 리스트가 전달되는데, 텐서는 Parameters를 가지며 경사 하강으로 최적화될 예정이므로 `requires_grad = True`를 기본으로 가진다. `training_loss.backward()`가 호출되면 옵티마이저에 전달된 파라미터인 `grad`가 그래프의 말단 노드들에 누적된다.

이제 SGD 옵티마이저에 필요한 모든 게 준비됐다. `optimizer.step()`이 호출되면 각 Parameter를 순회하며 `grad` 속성에 저장된 비율만큼 값을 조정한다. 깔끔하다.

훈련 루프를 다시 확인하자.

```
# In[13]:
```

```
def training_loop(n_epochs, optimizer, model, loss_fn, t_u_train, t_u_val,
                  t_c_train, t_c_val):
    for epoch in range(1, n_epochs + 1):
        t_p_train = model(t_u_train)
        loss_train = loss_fn(t_p_train, t_c_train)

        t_p_val = model(t_u_val)
        loss_val = loss_fn(t_p_val, t_c_val)

        optimizer.zero_grad()
        loss_train.backward()
        optimizer.step()

        if epoch == 1 or epoch % 1000 == 0:
            print(f"Epoch {epoch}, Training loss {loss_train.item():.4f},"
                  f" Validation loss {loss_val.item():.4f}")
```

←
← 개별 파라미터 대신
모델이 전달됐다.

← 손실 함수도 전달됐다. 곧 사용할 예정이다.

모델이 내부적으로 Parameters를 들고 있으니 명시적으로 model에 params를 전달하지 않게 된 것 외에는 바뀐 게 없다.

torch.nn에서 유용한 것이 하나 더 있는데 바로 손실 계산이다. nn 자체에 이미 일반적인 손실 함수가 들어 있다. 평균 제곱 오차^{Mean Square Error}를 의미하는 MSE는 nn.MSELoss인데 앞에서 정의했던 loss_fn과 동일하다. nn에 포함된 손실 함수는 nn.Module의 서브클래스이므로 인스턴스를 만들어 함수처럼 호출하면 된다. 직접 작성한 loss_fn을 바꿔보자.

```
# In[15]:
linear_model = nn.Linear(1, 1)
optimizer = optim.SGD(linear_model.parameters(), lr=1e-2)

training_loop(
    n_epochs = 3000,
    optimizer = optimizer,
    model = linear_model,
    loss_fn = nn.MSELoss(), ←—— 이제 직접 만든 손실 함수는 사용하지 않는다.
    t_u_train = t_un_train,
    t_u_val = t_un_val,
    t_c_train = t_c_train,
    t_c_val = t_c_val)

print()
print(linear_model.weight)
print(linear_model.bias)
```

```
# Out[15]:  
Epoch 1, Training loss 134.9599, Validation loss 183.1707  
Epoch 1000, Training loss 4.8053, Validation loss 4.7307  
Epoch 2000, Training loss 3.0285, Validation loss 3.0889  
Epoch 3000, Training loss 2.8569, Validation loss 3.9105  
  
Parameter containing:  
tensor([[5.4319]], requires_grad=True)  
Parameter containing:  
tensor([-17.9693], requires_grad=True)
```

훈련 루프에 들어가는 그 밖의 입력들은 이전과 동일하다. 심지어 결과도 기존과 같다. 만일 결과가 다르다면 둘 중 하나에 버그가 있을 것이다.

6.3 드디어 신경망

모델을 정의하고 훈련시키는 20여 줄의 코드에 알아야 할 내용이 상당히 많은, 제법 긴 여정이 있었다. 이제 훈련 과정에서 모호했던 마술 같은 영역이 실제 동작 방식으로 이해됐을 것이다. 덕분에 이전 문제가 복잡해져도 블랙 박스를 건드려보는 느낌이 아니라 코드를 자신의 것처럼 여기고 다룰 수 있게 되었다.

마지막으로 우리의 선형 모델 대신 신경망을 근사 함수로 사용하도록 바꿔보자. 앞에서 언급했지만 온도 보정 문제 자체가 근본적으로 선형이기 때문에 신경망 모델을 사용한다고 결과의 품질이 더 좋아지지는 않는다. 하지만 통제된 환경에서 선형 모델에서 신경망으로 도약해보는 것은 좋은 경험이고, 나중에 당황할 일도 적을 것이다.

6.3.1 선형 모델 대체하기

손실 함수를 포함한 나머지 모든 것은 그대로 두고 model만 다시 정의한다. 되도록 제일 단순한 신경망을 만들자. 선형 모듈 뒤에 활성화 함수를 달고 다른 선형 모듈에 연결하자. 첫 번째 선형 + 활성화층은 은닉층(hidden layer)이라고 불러왔다. 계층의 출력을 직접 관찰하는 대신 출력층의 입력으로 넣어줬기 때문이다. 지금은 모델의 입력과 출력 사이즈가 1이지만(입력 피쳐도 1이고 출력 피쳐도 1), 선형 모델의 출력 크기는 일반적으로 1보다는 크다. 앞에서 설명한 활성화 함수의 역할을 기억해보면 다양한 입력 범위에 대해 다양한 유닛이 응답할 수 있게 되므로 모델의 용량이 커진다고 볼 수 있다. 마지막 선형 계층은 활성화 함수의 출력을 받아 선형적으로 결합한 후 출력값을 만든다.

신경망을 기술하는 표준적인 방법은 없다. 그림 6.8에 두 가지 방법으로 나타내봤는데 왼쪽 그림이 우리의 모델이 어떤 모습일지 기본적인 수준에서 나타낸 것이고 오른쪽 그림은 리서치 논문이나 고급 문헌에서 볼 수 있는 형식이다. 파이토치가 제공하는 신경망 모듈을 블록으로 나타낸 다이어그램의 형태가 일반적이다(가끔 Tanh 같은 활성화층은 명시적으로 나타내지 않기도 한다). 그 외의 차이점으로는 왼쪽 그림은 입력이나 (중간) 결과를 주요 요소처럼 동그라미로 그린 반면, 오른쪽 그림은 단계별 연산이 도드라져 보인다.

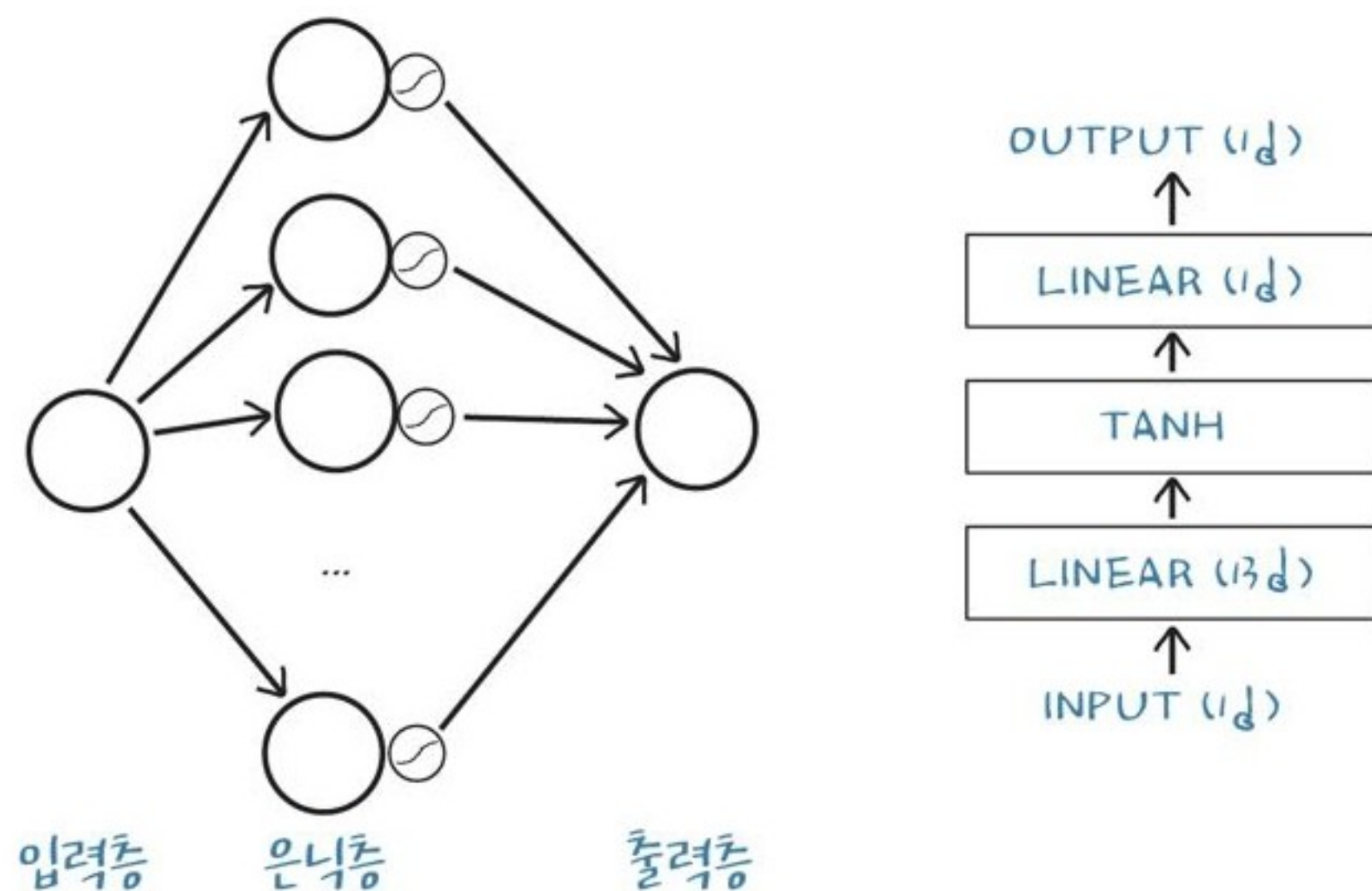


그림 6.8 두 가지 방식으로 그려본 우리의 간단한 신경망. 왼쪽은 초심자 버전이고 오른쪽은 높은 레벨의 버전이다.

nn은 nn.Sequential 컨테이너를 통해 모듈을 간단하게 결합시켜준다.

```
# In[16]:  
seq_model = nn.Sequential(  
    nn.Linear(1, 13),  ←—— 이전과 다른 텐서 형태를 가지려고 임의로 13을 골랐다.  
    nn.Tanh(),  
    nn.Linear(13, 1)) ←—— 다만 여기서는 앞의 13과 일치해야 한다.  
seq_model
```

```
# Out[16]:  
  
Sequential(  
  (0): Linear(in_features=1, out_features=13, bias=True)  
  (1): Tanh()  
  (2): Linear(in_features=13, out_features=1, bias=True)  
)
```

결과적으로 모델을 얻게 되며, 모델의 첫 번째 모듈은 nn.Sequential의 첫 번째 인자에 해당한다. 이 모듈의 중간 출력은 이어지는 다른 모듈에 전달되며 최종적으로 마지막 모듈에 의해 출력이 만들어진다. 모델은 한 개의 입력 피처로부터 13개의 은닉된 피처로 펼쳐지며, 결괏값을 tanh 활성 함수로 넘겨서 결과로 나온 13개의 숫자를 하나의 출력 피처로 만들기 위해 선형적으로 결합한다.

6.3.2 파라미터 살펴보기

`model.parameters()`를 호출하면 첫 번째와 두 번째 선형 모듈에서 `weight`와 `bias`를 모은다. 파라미터를 살펴보기 위해 차원 정보를 출력하는 게 좋다.

```
# In[17]:
```

```
[param.shape for param in seq_model.parameters()]
```

```
# Out[17]:
```

```
[torch.Size([13, 1]), torch.Size([13]), torch.Size([1, 13]), torch.Size([1])]
```

이 텐서가 최적화 함수로 전달될 텐서다. 앞서도 언급했지만 이제 `model.backward()`를 호출하면, `grad`에서 파라미터가 추출되고 옵티마이저는 `optimizer.step()` 호출 과정에서 적절하게 파라미터 값을 조정한다. 앞서의 선형 모델과 그렇게 큰 차이는 없는 것 같지 않은가? 둘 다 경사 하강을 통해 훈련시킬 수 있는 미분 가능한 모델이다.

`nn.Modules`의 파라미터에 대해 좀 더 알아보자. 여러 개의 서브 모듈로 만들어진 모델의 파라미터를 추적할 경우, 파라미터를 이름으로 식별가능하게 하면 매우 편리하다. 이를 위해 `named_parameters` 메소드가 존재한다.

```
# In[18]:  
for name, param in seq_model.named_parameters():  
    print(name, param.shape)
```

```
# Out[18]:  
0.weight torch.Size([13, 1])  
0.bias torch.Size([13])  
2.weight torch.Size([1, 13])  
2.bias torch.Size([1])
```

Sequential의 각 모듈 이름은 인자에서 모듈이 나타난 순서를 따른다. Sequential은 OrderedDict라는 인자도 받는데, 여기에 각 모듈의 이름을 정해서 전달해도 된다.⁴

```
# In[19]:
from collections import OrderedDict
seq_model = nn.Sequential(OrderedDict([
    ('hidden_linear', nn.Linear(1, 8)),
    ('hidden_activation', nn.Tanh()),
    ('output_linear', nn.Linear(8, 1))
]))

seq_model

# Out[19]:
Sequential(
  (hidden_linear): Linear(in_features=1, out_features=8, bias=True)
  (hidden_activation): Tanh()
  (output_linear): Linear(in_features=8, out_features=1, bias=True)
)
```


이렇게 이름을 붙이면 서브모듈을 더 쉽게 설명할 수 있다.

```
# In[20]:
for name, param in seq_model.named_parameters():
    print(name, param.shape)

# Out[20]:
hidden_linear.weight torch.Size([8, 1])
hidden_linear.bias torch.Size([8])
output_linear.weight torch.Size([1, 8])
output_linear.bias torch.Size([1])
```

이제 출력 결과가 훨씬 더 이해하기 쉬워졌다. 그렇다고 신경망을 통해 지나가는 데이터의 흐름을 더 유연하게 해주는 것은 아니다. 우리의 신경망이 만든 경로는 순수하게 연속적인 경로를 만들어 내는데 이런 점에서 `nn.Sequential`이라는 이름은 매우 적절하다. `nn.Module`를 서브클래스로 만들어 입력 데이터를 완벽하게 제어하고 처리하는 방법은 8장에서 곧 살펴볼 것이다.

서브 모듈을 속성처럼 사용해 특정 Parameter에 접근할 수도 있다.

```
# In[21]:  
seq_model.output_linear.bias  
  
# Out[21]:  
Parameter containing:  
tensor([-0.0173], requires_grad=True)
```

파라미터를 추적하거나 기울기를 파악하기에 유용한 방법이다. 예를 들어 이 장의 앞에서 했던 것처럼 훈련시키는 동안 기울기를 모니터링하는 경우 유용하다. 은닉층의 선형 영역에서 weight의 기울기를 출력하고 싶다고 가정하자. 이럴 때에는 새로운 신경망 모델을 위한 훈련 루프를 돌리고 마지막 에포크 후에 만들어진 기울기를 보면 된다.

```
# In[22]:
```

```
optimizer = optim.SGD(seq_model.parameters(), lr=1e-3)
```

← 안정성을 위해 학습률을
조금 떨어뜨렸다.

```
training_loop(
```

```
    n_epochs = 5000,  
    optimizer = optimizer,  
    model = seq_model,  
    loss_fn = nn.MSELoss(),  
    t_u_train = t_un_train,  
    t_u_val = t_un_val,  
    t_c_train = t_c_train,  
    t_c_val = t_c_val)
```

```
print('output', seq_model(t_un_val))
```

```
print('answer', t_c_val)
```

```
print('hidden', seq_model.hidden_linear.weight.grad)
```



```
# Out[22]:
Epoch 1, Training loss 182.9724, Validation loss 231.8708
Epoch 1000, Training loss 6.6642, Validation loss 3.7330
Epoch 2000, Training loss 5.1502, Validation loss 0.1406
Epoch 3000, Training loss 2.9653, Validation loss 1.0005
Epoch 4000, Training loss 2.2839, Validation loss 1.6580
Epoch 5000, Training loss 2.1141, Validation loss 2.0215
output tensor([[ -1.9930],
               [20.8729]], grad_fn=<AddmmBackward>)
answer tensor([[ -4.],
               [21.]])
hidden tensor([[ 0.0272],
               [ 0.0139],
               [ 0.1692],
               [ 0.1735],
               [-0.1697],
               [ 0.1455],
               [-0.0136],
               [-0.0554]])
```

6.3.3 선형 모델과 비교하기

이제 모든 데이터에 대한 우리의 모델이 단순 선형과 어떤 차이가 있는지 평가할 수 있다.

```
# In[23]:  
from matplotlib import pyplot as plt  
  
t_range = torch.arange(20., 90.).unsqueeze(1)  
  
fig = plt.figure(dpi=600)  
plt.xlabel("Fahrenheit")  
plt.ylabel("Celsius")  
plt.plot(t_u.numpy(), t_c.numpy(), 'o')  
plt.plot(t_range.numpy(), seq_model(0.1 * t_range).detach().numpy(), 'c-')  
plt.plot(t_u.numpy(), seq_model(0.1 * t_u).detach().numpy(), 'kx')
```

그림 6.9에 결과가 그려져 있다. 잘 보면 5장에서 다뤘던 것과 같이 신경망이 노이즈까지 포함해서 측정값을 과도하게 따라가는 과적합 성향을 보임을 알 수 있다. 작디 작은 신경망임에도 측정값이 얼마 되지 않기 때문에 상대적으로 적합에 필요한 양보다 많은 파라미터를 가지고 있는 셈이다. 그래도 결과적으로는 나쁘지 않다.

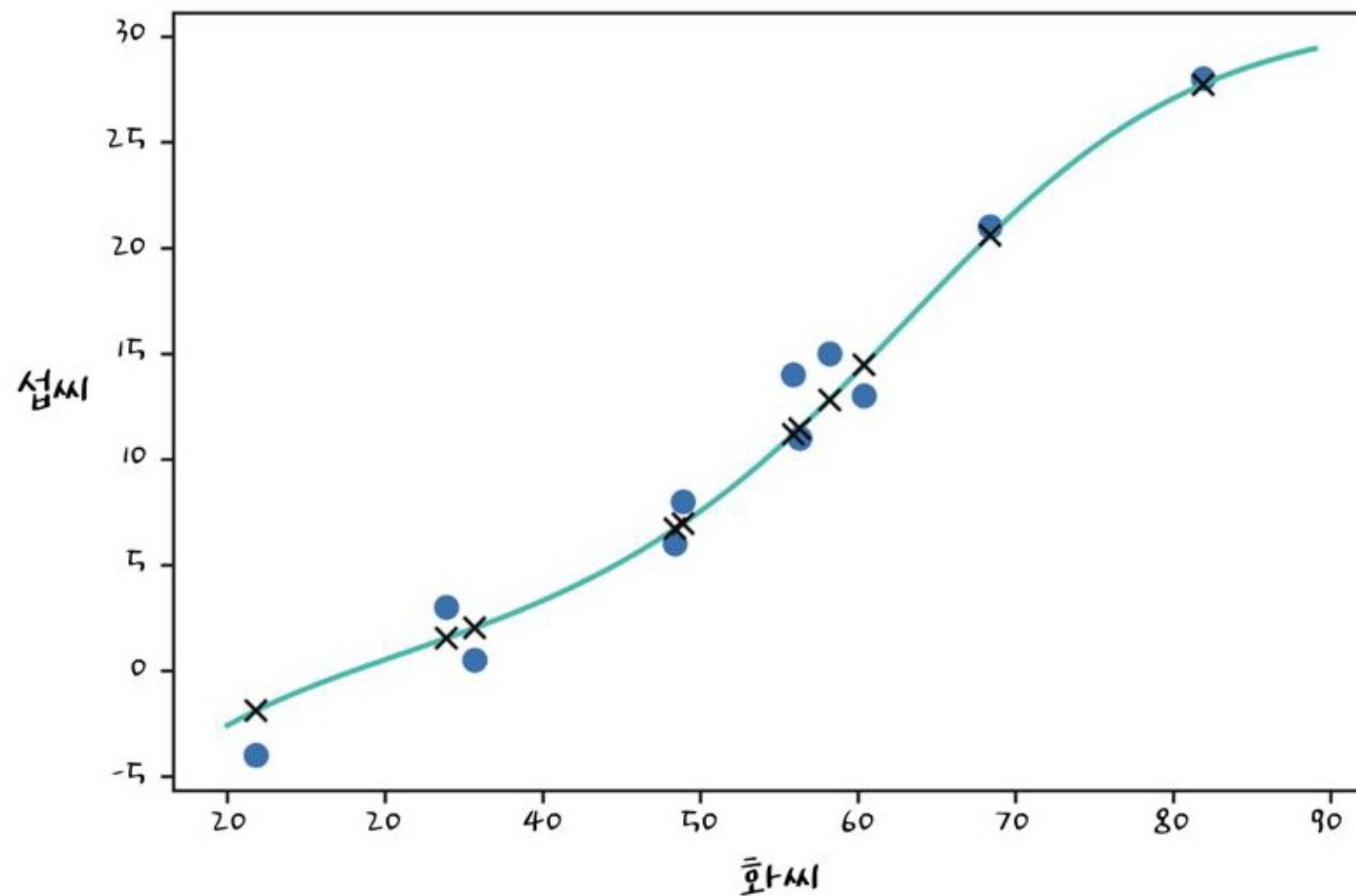


그림 6.9 입력 데이터는 ○로, 출력 데이터는 ×로 표기한 신경망 모델 그래프. 연속된 선이 샘플 사이에서 모델의 행동을 보여준다.

6.4 결론

매우 간단한 문제를 다루기는 했지만 5장과 6장을 통해 꽤 많은 내용을 다뤘다. 미분 가능한 모델을 만들고 경사 하강을 통해 처음에는 자동미분으로, 두 번째는 nn으로 훈련도 진행했다. 이제 신경망 내부가 어떻게 동작하는지 확실하게 이해했다고 자신해도 좋다. 지금까지 배우면서 알게 된 파이토치를 더 잘 다루고 싶어지기 바란다!

6.6 핵심 요약

- 신경망은 특정 문제를 풀기 위해 적합한 형태로 스스로를 최적화할 수 있다.
- 신경망은 모델의 어떤 파라미터에 대해서라도 손실값의 미분에 대해 쉽게 접근할 수 있게 하여 파라미터가 매우 효율적으로 진화하도록 만들어준다. 감사하게도 파이토치의 자동화된 미분 엔진을 통해 이러한 미분 과정을 쉽게 사용할 수 있다.
- 선형 변환을 둘러싼 활성화 함수 덕분에 신경망은 상당히 비선형적인 함수에 대해서도 근사할 수 있으며 동시에 최적화하기에 충분한 수준으로 단순한 형태를 유지할 수 있다.
- nn 모듈과 표준 텐서 라이브러리는 신경망을 만들기 위한 모든 빌딩 블록을 제공한다.
- 과적합을 감지하기 위해 훈련셋을 검증셋과 분리해서 유지하는 것은 필수적이다. 과적합을 방지하는 방법은 다양하다. 더 많은 데이터를 구하거나, 데이터가 더욱 다양한 값을 가지게 한다면, 더 단순한 모델을 사용하는 방법 등을 먼저 시도해볼 만하다.
- 데이터과학을 하는 사람이라면 항상 데이터를 차트에 그려봐야 한다.